Chapitre 30

Polynômes et transformée rapide de Fourier

La méthode directe pour additionner deux polynômes de degré n requiert un temps $\Theta(n)$, tandis que la méthode directe pour multiplier deux polynômes de degré n prend un temps $\Theta(n^2)$. Dans ce chapitre, nous allons montrer comment la transformée rapide de Fourier, ou FFT (Fast Fourier Transform), peut ramener le temps de multiplication de deux polynômes à $\Theta(n \lg n)$.

Les transformées de Fourier, dont la FFT, servent essentiellement à faire du traitement de signaux. On a un signal donné dans un ***domaine des temps***, à savoir une fonction qui associe un temps à une amplitude. L'analyse de Fourier permet d'exprimer le signal sous la forme d'une somme pondérée de sinusoïdes à décalage de phase et à fréquence variable. Les poids et phases associés aux fréquences caractérisent le signal dans le ***domaine des fréquences***. Le traitement du signal est un domaine très riche qui a suscité une littérature de haut niveau, dont les notes du chapitres donnent un aperçu.

© Dunod – La photocopie non autorisée est un délit

a) Polynômes

Un ***polynôme*** en x sur un corps algébrique K est la représentation d'une fonction $A(x)$ sous la forme d'une somme formelle :

$$A(x) = \sum_{j=0}^{n-1} a_j x^j .$$

Scanned with CamScanner

Les valeurs $a_0, a_1, \ldots, a_{n-1}$ sont appelées ***coefficients*** du polynôme. Les coefficients appartiennent au corps K, qui est généralement le corps $\mathbf{C}$ des nombres complexes. Un polynôme $A(x)$ est de ***degré*** k si son coefficient non nul de plus haut rang est a_k. Un entier strictement supérieur au degré d'un polynôme est un ***majorant du degré*** de ce polynôme. Le degré d'un polynôme de degré borné par n peut donc être n'importe quel entier compris entre 0 et $n - 1$ inclus.

Il est possible de définir une grande variété d'opérations sur les polynômes. Pour ***l'addition des polynômes***, si $A(x)$ et $B(x)$ sont des polynômes de degrés majorés par n, leur ***somme*** est un polynôme $C(x)$, de degré également borné par n, tel que $C(x) = A(x) + B(x)$ pour tout x appartenant au corps sous-jacent. Autrement dit, si

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

et

$$B(x) = \sum_{j=0}^{n-1} b_j x^j ,$$

alors

$$C(x) = \sum_{j=0}^{n-1} c_j x^j ,$$

où $c_j = a_j + b_j$ pour $j = 0, 1, \ldots, n - 1$. Par exemple, si l'on a les polynômes $A(x) = 6x^3 + 7x^2 - 10x + 9$ et $B(x) = -2x^3 + 4x - 5$, alors $C(x) = 4x^3 + 7x^2 - 6x + 4$.

Pour la ***multiplication des polynômes***, si $A(x)$ et $B(x)$ sont des polynômes de degrés majorés par n, leur ***produit*** $C(x)$ est un polynôme, de degré borné par $2n - 1$, tel que $C(x) = A(x)B(x)$ pour tout x appartenant au corps sous-jacent. Vous avez sans doute déjà multiplié des polynômes, en multipliant chaque terme de $A(x)$ par chaque terme de $B(x)$ et en combinant les termes de puissances égales. Par exemple, on peut multiplier $A(x) = 6x^3 + 7x^2 - 10x + 9$ et $B(x) = -2x^3 + 4x - 5$ de la manière suivante :

$$\begin{array}{rrrrrrr}
 & 6x^3 & + & 7x^2 & - & 10x & + & 9 \\
- & 2x^3 & & & + & 4x & - & 5 \\
\hline
 & -12x^6 & - & 14x^5 & + & 40x^4 & - & 18x^3 \\
 & & - & 30x^3 & - & 35x^2 & + & 50x & - & 45 \\
\hline
 & -12x^6 & - & 14x^5 & + & 44x^4 & - & 20x^3 & - & 75x^2 & + & 86x & - & 45
\end{array}$$

Le produit $C(x)$ peut aussi s'exprimer sous la forme

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j , \tag{30.1}$$

Scanned with CamScanner

où

$$c_j = \sum_{k=0}^{j} a_k b_{j-k} . \tag{30.2}$$

Notez que degré(C) = degré(A) + degré(B), ce qui implique que, si A est un polynôme de degré borné par n_a et B un polynôme de degré borné par n_b, alors C est un polynôme de degré borné par $n_a + n_b - 1$. Comme un polynôme de degré borné par k est aussi un polynôme de degré borné par k+1, on dit normalement que le polynôme produit C est un polynôme de degré borné par $n_a + n_b$.

b) Présentation du chapitre

La section 30.1 montre deux façons de représenter les polynômes : la représentation par coefficients et la représentation par paires point-valeur. Les méthodes directes de multiplication de polynômes (équations (30.1) et (30.2)) demandent un temps $\Theta(n^2)$ quand les polynômes sont représentés par leurs coefficients, mais seulement $\Theta(n)$ seulement quand ils sont représentés par des paires point-valeur. On peut, toutefois, multiplier des polynômes, représentés par leurs coefficients, en un temps $\Theta(n \lg n)$, en effectuant une conversion entre les deux représentations. Pour comprendre ce mécanisme, il faut d'abord étudier les racines complexes de l'unité, ce que nous ferons à la section 30.2. Ensuite, nous utiliserons la FFT et sa réciproque, également décrite dans la section 30.2, pour effectuer les conversions. La section 30.3 montre comment implémenter efficacement la FFT tant dans le modèle sériel que dans le modèle parallèle.

Ce chapitre fait beaucoup appel aux nombres complexes et le symbole i sera employé exclusivement pour représenter $\sqrt{-1}$.

30.1 REPRÉSENTATION DES POLYNÔMES

La représentation par coefficients et la représentation par paires point-valeur sont, en un sens, équivalentes : tout polynôme représenté par un ensemble de paires point-valeur est associé à un et un seul polynôme représenté par des coefficients. Dans cette section, on introduit les deux représentations et on montre comment elles peuvent se combiner pour permettre de multiplier deux polynômes de degré borné par n en un temps $\Theta(n \lg n)$.

c) Représentation par coefficients

Une ***représentation par coefficients*** d'un polynôme $A(x) = \sum_{j=0}^{n-1} a_j x^j$ de degré borné par n est un vecteur de coefficients $a = (a_0, a_1, \ldots, a_{n-1})$. Dans les équations matricielles de ce chapitre, on traitera généralement les vecteurs comme des vecteurs colonne.

© Dunod - La photocopie non autorisée est un délit

Scanned with CamScanner

La représentation par coefficients est commode pour certaines opérations sur les polynômes. Par exemple, l'opération ***d'évaluation*** du polynôme $A(x)$ en un point donné x_0 consiste à calculer la valeur de $A(x_0)$. L'évaluation nécessite un temps $\Theta(n)$ si l'on utilise la ***règle de Horner*** :

$$A(x_0) = a_0 + x_0(a_1 + x_0(a_2 + \cdots + x_0(a_{n-2} + x_0(a_{n-1}))\cdots)) .$$

De même, l'addition de deux polynômes représentés par les vecteurs de coefficients $a = (a_0, a_1, \ldots, a_{n-1})$ et $b = (b_0, b_1, \ldots, b_{n-1})$ prend un temps $\Theta(n)$: il suffit de calculer le vecteur de coefficients $c = (c_0, c_1, \ldots, c_{n-1})$, où $c_j = a_j + b_j$ pour $j = 0, 1, \ldots, n-1$.

Considérons à présent la multiplication de deux polynômes, de degré borné par n, $A(x)$ et $B(x)$ représentés par leurs coefficients. Si l'on utilise la méthode décrite aux équations (30.1) et (30.2), la multiplication des polynômes requiert $\Theta(n^2)$, puisque chaque coefficient du vecteur a doit être multiplié par chaque coefficient du vecteur b. La multiplication des polynômes représentés par leur coefficients paraît beaucoup plus complexe que l'évaluation d'un polynôme ou l'addition de polynômes. Le vecteur de coefficients résultant c, donné par l'équation (30.2), est également appelé ***convolution*** des vecteurs d'entrées a et b, ce qu'on note par $c = a \otimes b$. Comme la multiplication des polynômes et le calcul des convolutions sont des problèmes calculatoires fondamentaux d'une importance pratique considérable, ce chapitre insiste sur les algorithmes efficaces permettant de les résoudre.

d) Représentation par paires point-valeur

Une ***représentation par paires point-valeur*** d'un polynôme $A(x)$ de degré borné par n est un ensemble de n points du plan, représentés par leurs ***coordonnées***

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$$

telles que les x_k sont tous distincts et que

$$y_k = A(x_k) \tag{30.3}$$

pour $k = 0, 1, \ldots, n-1$. Un même polynôme possède plusieurs représentations par paires point-valeur, puisqu'un ensemble quelconque de n points distincts $x_0, x_1, \ldots, x_{n-1}$ peut servir de base à la représentation.

Le calcul d'une représentation par paires point-valeur d'un polynôme représenté par ses coefficients est simple dans son principe, puisqu'il suffit de choisir n abscisses distinctes $x_0, x_1, \ldots, x_{n-1}$ et d'évaluer ensuite $A(x_k)$ pour $k = 0, 1, \ldots, n-1$. Avec la méthode de Horner, cette évaluation pour n points prend $\Theta(n^2)$. Nous verrons plus tard que, si l'on choisit intelligemment les x_k, le temps requis pour ce calcul peut être ramené à $\Theta(n \lg n)$.

L'inverse de l'évaluation, à savoir la détermination des coefficients d'un polynôme à partir d'une représentation par paires point-valeur, s'appelle ***interpolation***. Le théorème suivant montre que l'interpolation est bien définie quand le degré du polynôme d'interpolation souhaité doit être borné par le nombre de paires point-valeur donné.

Scanned with CamScanner

Théorème 30.1 (Unicité d'un polynôme d'interpolation) *Pour un ensemble* $\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$ *quelconque de n paires point-valeur telles que tous les* x_k *soient distincts, il existe un unique polynôme* $A(x)$ *de degré inférieur à n tel que* $y_k = A(x_k)$ *pour* $k = 0, 1, \ldots, n-1$.

Démonstration : La démonstration est basée sur l'existence de l'inverse d'une certaine matrice. L'équation (30.3) est équivalente à l'équation matricielle

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix} . \tag{30.4}$$

La matrice de gauche est notée $V(x_0, x_1, \ldots, x_{n-1})$ et est connue sous le nom de matrice de Vandermonde. D'après l'exercice 28.1.11, cette matrice a pour déterminant

$$\prod_{0 \le j < k \le n-1} (x_k - x_j) ,$$

et donc, d'après le théorème 28.5, elle est inversible (c'est-à-dire non singulière) si les x_k sont distincts. Donc, on peut trouver des coefficients a_j uniques à partir d'une quelconque représentation par paires point-valeur :

$$a = V(x_0, x_1, \ldots, x_{n-1})^{-1} y .$$

□

La démonstration du théorème 30.1 décrit un algorithme d'interpolation basé sur la résolution de l'ensemble (30.4) d'équations linéaires. A l'aide des algorithmes de décomposition LU du chapitre 28, on peut résoudre ces équations en temps $O(n^3)$.

Un algorithme plus rapide pour l'interpolation sur n points est basé sur la ***formule de Lagrange*** :

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \ne k} (x - x_j)}{\prod_{j \ne k} (x_k - x_j)} . \tag{30.5}$$

On pourra vérifier que le membre droit de l'équation (30.5) est un polynôme de degré borné par n qui vérifie $A(x_i) = y_i$ pour tout i. L'exercice 30.1.5 vous demandera de calculer les coefficients de A à l'aide de la formule de Lagrange en temps $\Theta(n^2)$.

Ainsi, l'évaluation et l'interpolation de n points sont des opérations inverses bien définies qui permettent de passer de la représentation par coefficients d'un polynôme à sa représentation par paires point-valeur[1]. Les algorithmes décrits ci-dessus pour ces problèmes s'exécutent en $\Theta(n^2)$.

---

(1) Il est bien connu que l'interpolation est un problème subtil quant à sa stabilité numérique. Bien que les approches décrites ici soient mathématiquement correctes, de petites différences dans les données ou des erreurs d'arrondi au cours du calcul peuvent provoquer de grandes différences dans le résultat.

© Dunod - La photocopie non autorisée est un délit

Scanned with CamScanner

La représentation par paires point-valeur est efficace pour de nombreuses opérations sur les polynômes. Pour l'addition, si $C(x) = A(x) + B(x)$, alors $C(x_k) = A(x_k) + B(x_k)$ pour tout x_k. Plus précisément, si l'on a une représentation par paires point-valeur pour A,

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\} ,$$

et pour B,

$$\{(x_0, y'_0), (x_1, y'_1), \ldots, (x_{n-1}, y'_{n-1})\}$$

(notez que A et B sont évalués pour les *mêmes* n points), alors une représentation par paires point-valeur de C sera

$$\{(x_0, y_0 + y'_0), (x_1, y_1 + y'_1), \ldots, (x_{n-1}, y_{n-1} + y'_{n-1})\} .$$

Le temps nécessaire à l'addition de deux polynômes de degré borné par n, dans la représentation par paires point-valeur, est donc $\Theta(n)$.

La représentation par paires point-valeur est également pratique pour la multiplication des polynômes. Si $C(x) = A(x)B(x)$, alors $C(x_k) = A(x_k)B(x_k)$ pour tout x_k et l'on peut multiplier, point par point, une représentation par paires point-valeur de A par une représentation par paires point-valeur de B pour obtenir une représentation par paires point-valeur de C. Il faut, toutefois, résoudre le problème induit par degré(C) = degré(A) + degré(B) ; si A et B sont de degré borné par n, alors C est de degré borné par $2n$. Une représentation par paires point-valeur normale de A et B se compose de n paires pour chaque polynôme. En multipliant ces paires deux à deux, on obtient n paires point-valeur ; mais il faut $2n$ paires pour interpoler un polynôme unique C de degré borné par $2n$. (Voir exercice 30.1.4.) Il faut donc prendre des représentations « étendues » pour A et pour B, composées de $2n$ coordonnées chacune. Étant donnée une représentation étendue de A par paires point-valeur,

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{2n-1}, y_{2n-1})\} ,$$

et une représentation étendue correspondante pour B,

$$\{(x_0, y'_0), (x_1, y'_1), \ldots, (x_{2n-1}, y'_{2n-1})\} ,$$

C aura pour représentation étendue

$$\{(x_0, y_0 y'_0), (x_1, y_1 y'_1), \ldots, (x_{2n-1}, y_{2n-1} y'_{2n-1})\} .$$

Étant donnés deux polynômes représentés par des paires point-valeur étendues, on voit que le temps pris pour les multiplier et obtenir une représentation par valeurs du résultat est $\Theta(n)$; c'est très inférieur au temps nécessaire pour multiplier deux polynômes qui sont donnés par leurs coefficients.

Enfin, comment évaluer en un nouveau point un polynôme qui est représenté par des paires point-valeur ? Pour ce problème, il n'existe apparemment pas de méthode plus simple que de calculer les coefficients du polynôme par interpolation, puis de l'évaluer au nouveau point.

Scanned with CamScanner

e) Multiplication rapide de polynômes donnés par leurs coefficients

Est-il possible d'utiliser la méthode de multiplication à temps linéaire, employée pour des polynômes donnés par des paires point-valeur, pour multiplier des polynômes donnés par leurs coefficients ? La réponse est déterminée par notre capacité à convertir rapidement un polynôme de la représentation par coefficients vers la représentation par paires point-valeur (évaluation) et vice versa (interpolation).

On peut utiliser n'importe quel point comme point d'évaluation ; mais en choisissant soigneusement les points d'évaluation, on peut effectuer la conversion entre les deux représentations en seulement $\Theta(n \lg n)$. Comme nous le verrons à la section 30.2, si l'on choisit les « racines complexes de l'unité » comme points d'évaluation, on peut produire une représentation par paires point-valeur en prenant la transformée discrète de Fourier (DFT) d'un vecteur de coefficients. L'opération inverse, l'interpolation, peut s'effectuer en prenant la « DFT inverse » de paires point-valeur pour calculer un vecteur de coefficients. La section 30.2 montrera comment la transformée rapide de Fourier effectue la transformation discrète de Fourier et son inverse en $\Theta(n \lg n)$.

La figure 30.1 illustre graphiquement cette stratégie. Un petit détail à prendre en compte concerne les degrés. Le produit de deux polynômes de degré borné par n est un polynôme de degré borné par $2n$. Avant d'évaluer les polynômes A et B, il faut donc commencer par doubler leurs bornes de degré en ajoutant n coefficients de poids fort ayant la valeur 0. Comme les vecteurs ont $2n$ éléments, on utilise les « racines $(2n)$ièmes complexes de l'unité », qui sont représentées par les termes ω_{2n} sur la figure 30.1.

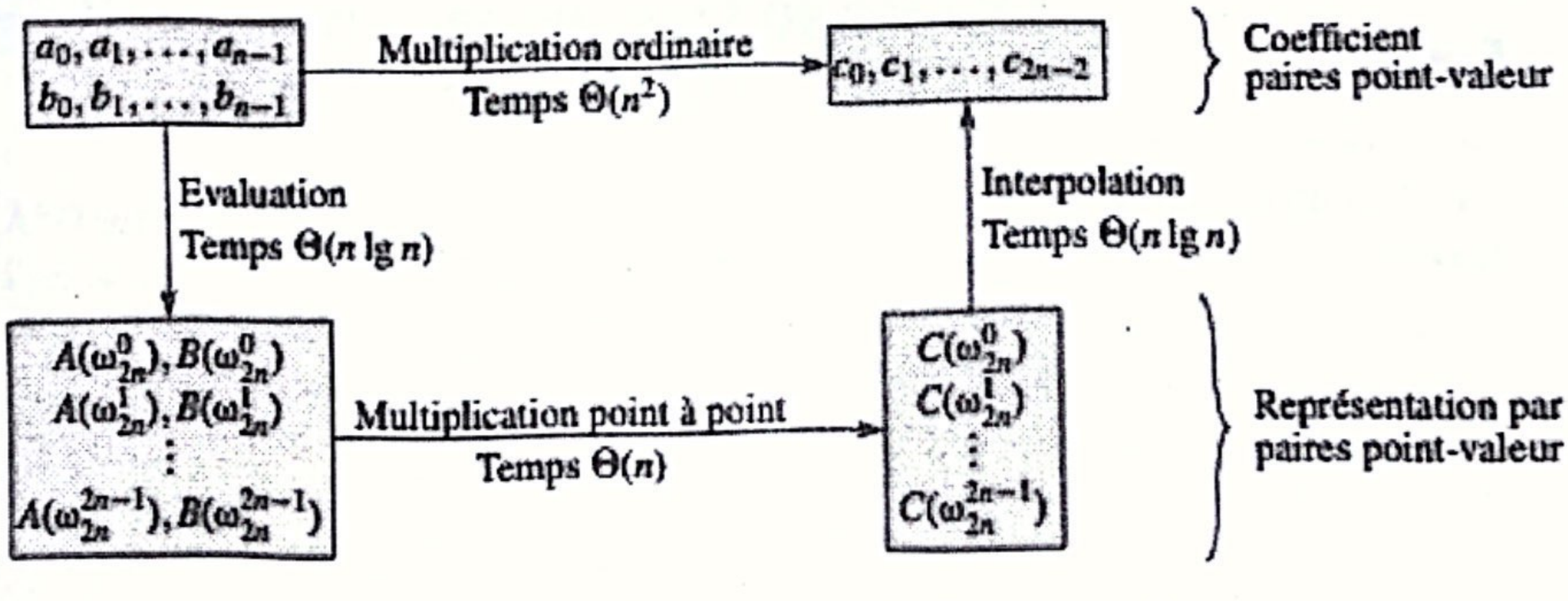


Figure 30.1 Schéma d'un processus efficace de multiplication de polynômes. Les représentations du haut sont par coefficients et celles du bas sont par paires point-valeur. Les flèches de la gauche vers la droite correspondent à l'opération de multiplication. Les termes ω_{2n} sont les racines $(2n)$ièmes complexes de l'unité.

Étant donnée la FFT, on a la procédure en $\Theta(n \lg n)$ suivante pour multiplier deux polynômes $A(x)$ et $B(x)$, de degré borné par n, où les représentations sont par coefficients en entrée comme en sortie. On suppose que n est une puissance de 2 ; cette

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

contrainte peut toujours être respectée en ajoutant des coefficients nuls dans les rangs supérieurs.

1) *Doublage de la borne :* On crée des représentations par coefficients de $A(x)$ et $B(x)$ pour en faire des polynômes de degré borné par $2n$, en ajoutant à chacun n coefficients nuls dans les rangs supérieurs.

2) *Évaluation :* On calcule les représentations par paires point-valeur de $A(x)$ et $B(x)$, de longueur $2n$, en appliquant deux fois la FFT d'ordre $2n$. Ces représentations contiennent les valeurs des deux polynômes pour les racines $(2n)$ièmes de l'unité.

3) *Multiplication point à point :* On calcule une représentation par paires point-valeur pour le polynôme $C(x) = A(x)B(x)$, en multipliant ces valeurs point à point. Cette représentation contient la valeur de $C(x)$ pour chaque racine $(2n)$ième de l'unité.

4) *Interpolation :* On crée la représentation par coefficients du polynôme $C(x)$ en appliquant une seule fois la FFT à $2n$ paires point-valeur pour calculer l'inverse de la transformée discrète de Fourier.

Les étapes (1) et (3) nécessitent $\Theta(n)$ et les étapes (2) et (4) $\Theta(n \lg n)$. Donc, une fois que nous aurons montré comment utiliser la FFT, nous aurons prouvé le théorème suivant :

Théorème 30.2 *Le produit de deux polynômes de degré borné par n peut être calculé en $\Theta(n \lg n)$, si l'entrée et la sortie sont représentées par leur coefficients.*

Exercices

30.1.1 Multiplier les polynômes $A(x) = 7x^3 - x^2 + x - 10$ et $B(x) = 8x^3 - 6x + 3$ à l'aide des équations (30.1) et (30.2).

30.1.2 L'évaluation d'un polynôme $A(x)$ de degré borné par n en un point donné x_0 peut aussi être effectuée en divisant $A(x)$ par le polynôme $(x - x_0)$ pour obtenir un polynôme quotient $q(x)$ de degré borné par $n - 1$ et un reste r, tel que

$$A(x) = q(x)(x - x_0) + r .$$

On voit clairement que $A(x_0) = r$. Montrer comment calculer le reste r et les coefficients de $q(x)$ en temps $\Theta(n)$ à partir de x_0 et des coefficients de A.

30.1.3 Déduire une représentation par paires point-valeur de $A^{\text{inv}}(x) = \sum_{j=0}^{n-1} a_{n-1-j}x^j$ à partir d'une représentation par paires point-valeur de $A(x) = \sum_{j=0}^{n-1} a_j x^j$, en supposant qu'aucun des points n'est 0.

Scanned with CamScanner

30.1.4 Prouver qu'il faut n paires point-valeur distinctes pour définir sans ambiguïté un polynôme de degré borné par n ; en d'autres termes, si l'on a moins de n paires point-valeur, elles ne peuvent pas définir un polynôme unique de degré borné par n. (*conseil :* En utilisant le théorème 30.1, que peut-on dire d'un ensemble de $n - 1$ paires point-valeur auquel on ajoute une paire supplémentaire choisie arbitrairement ?)

30.1.5 Montrer comment utiliser l'équation (30.5) pour interpoler en $\Theta(n^2)$. (*conseil :* Commencer par calculer la représentation par coefficients du polynôme $\prod_j (x - x_j)$, puis diviser par $(x - x_k)$ comme numérateur de chaque terme ; voir exercice 30.1.2.) Chacun des n dénominateurs devra être calculé en temps $O(n)$

30.1.6 Expliquer l'inconvénient de l'approche « directe » pour la division de polynômes basée sur la représentation par paires point-valeur. Étudier séparément le cas où la division tombe juste et celui où le reste n'est pas nul.

30.1.7 On considère deux ensembles A et B, contenant chacun n entiers compris entre 0 et $10n$. On souhaite calculer la ***somme cartésienne*** de A et B, définie par

$$C = \{x + y : x \in A \text{ et } y \in B\} .$$

Notez que les entiers de C sont compris entre 0 et $20n$. On veut trouver les éléments de C et le nombre de fois que chaque élément de C est produit par une somme d'éléments de A et B. Montrer qu'on peut résoudre ce problème en temps $O(n \lg n)$. (*conseil :* Représenter A et B sous la forme de polynômes de degré $10n$ au plus.)

30.2 TRANSFORMÉE DISCRÈTE DE FOURIER ET TRANSFORMÉE RAPIDE DE FOURIER

À la section 30.1, nous affirmions que, si l'on utilisait les racines complexes de l'unité, on pouvait évaluer et interpoler les polynômes en temps $\Theta(n \lg n)$. Dans cette section, nous définissons les racines complexes de l'unité et étudions leurs propriétés, nous définissons la transformée discrète de Fourier (DFT) et montrons comment la FFT peut calculer la DFT et son inverse en seulement $\Theta(n \lg n)$.

a) Racines complexes de l'unité

Une ***racine nième complexe de l'unité*** est un nombre complexe ω tel que

$$\omega^n = 1 .$$

Il existe exactement n racines $n$ièmes complexes de l'unité ; $e^{2\pi ik/n}$ pour $k = 0, 1, \ldots, n - 1$. Pour interpréter cette formule, on utilise la définition de l'exponentielle d'un nombre complexe :

$$e^{iu} = \cos(u) + i \sin(u) .$$

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

La figure 30.2 montre que les n racines complexes de l'unité sont espacées de façon régulière autour du cercle de rayon unitaire centré à l'origine du plan complexe. La valeur

$$\omega_n = e^{2\pi i/n} \tag{30.6}$$

est appelée ***racine nième principale de l'unité*** ; toutes les autres racines complexes de l'unité sont des puissances de ω_n. (2)

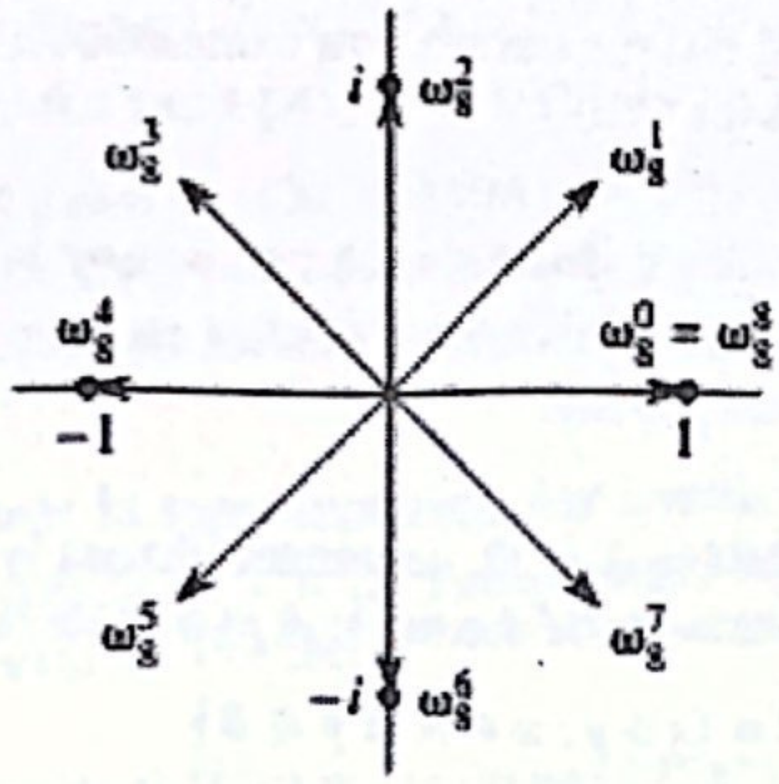


Figure 30.2 Les valeurs de $\omega_8^0, \omega_8^1, \ldots, \omega_8^7$ dans le plan complexe, où $\omega_8 = e^{2\pi i/8}$ est la racine 8ième principale de l'unité.

Les n racines $n$ièmes de l'unité,

$$\omega_n^0, \omega_n^1, \ldots, \omega_n^{n-1} ,$$

forment un groupe pour la multiplication (voir section 31.3). Ce groupe a la même structure que le groupe additif $(\mathbb{Z}_n, +)$ modulo n, puisque $\omega_n^n = \omega_n^0 = 1$ implique $\omega_n^j \omega_n^k = \omega_n^{j+k} = \omega_n^{(j+k) \bmod n}$. De même, $\omega_n^{-1} = \omega_n^{n-1}$. Les lemmes suivants donnent les propriétés essentielles des racines $n$ièmes de l'unité.

Lemme 30.3 (Lemme de l'annulation) *Quels que soient les entiers $n \geqslant 0$, $k \geqslant 0$ et $d > 0$,*

$$\omega_{dn}^{dk} = \omega_n^k . \tag{30.7}$$

Démonstration : Le lemme se déduit directement de l'équation (30.6), puisque

$$\begin{aligned} \omega_{dn}^{dk} &= \left(e^{2\pi i/dn}\right)^{dk} \\ &= \left(e^{2\pi i/n}\right)^{k} \\ &= \omega_n^k . \end{aligned}$$

❑

(2) Maints auteurs définissent ω_n différemment : $\omega_n = e^{-2\pi i/n}$. Cette autre définition tend à s'imposer dans les applications du traitement du signal. Les mathématiques sous-jacentes sont essentiellement les mêmes, quelle que soit la définition de ω_n.

Scanned with CamScanner

Corollaire 30.4 *Pour tout entier pair $n > 0$,*

$$\omega_n^{n/2} = \omega_2 = -1 .$$

Démonstration : La démonstration est laissée en exercice (voir exercice 30.2.1. ❑

Lemme 30.5 (Lemme de la bipartition) *Si $n > 0$ est pair, les carrés des n racines $n$ièmes de l'unité sont les $n/2$ racines $(n/2)$ièmes de l'unité.*

Démonstration : D'après le lemme de l'annulation, on a $(\omega_n^k)^2 = \omega_{n/2}^k$, pour tout entier non négatif k. Notez que, si l'on élève au carré toutes les racines $n$ièmes de l'unité, chaque racine $(n/2)$ième de l'unité est obtenue exactement deux fois, puisque

$$\begin{aligned}(\omega_n^{k+n/2})^2 &= \omega_n^{2k+n} \\ &= \omega_n^{2k}\omega_n^n \\ &= \omega_n^{2k} \\ &= (\omega_n^k)^2 .\end{aligned}$$

Donc, ω_n^k et $\omega_n^{k+n/2}$ ont le même carré. Cette propriété peut également être démontrée à l'aide du corollaire 30.4, puisque $\omega_n^{n/2} = -1$ implique $\omega_n^{k+n/2} = -\omega_n^k$ et donc $(\omega_n^{k+n/2})^2 = (\omega_n^k)^2$. ❑

Comme nous le verrons, le lemme de la bipartition est essentiel pour notre approche diviser-pour-régner de la conversion entre représentation par coefficients et représentation par paires point-valeur des polynômes, car elle garantit que les sous-problèmes récursifs seront deux fois moins grands.

Lemme 30.6 (Lemme de la sommation) *Pour tout entier $n \geq 1$ et tout entier k non nul et non divisible par n,*

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = 0 .$$

Démonstration : L'équation (A.5) s'appliquant aussi aux valeurs complexes, on a donc

$$\begin{aligned}\sum_{j=0}^{n-1} (\omega_n^k)^j &= \frac{(\omega_n^k)^n - 1}{\omega_n^k - 1} = \frac{(\omega_n^n)^k - 1}{\omega_n^k - 1} \\ &= \frac{(1)^k - 1}{\omega_n^k - 1} = 0 .\end{aligned}$$

On impose à k de ne pas être divisible par n pour assurer que le dénominateur n'est pas nul. En effet, $\omega_n^k = 1$ seulement quand k est divisible par n. ❑

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

b) Transformée discrète de Fourier

On se souvient que notre objectif est d'évaluer un polynôme

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

de degré borné par n sur les points $\omega_n^0, \omega_n^1, \omega_n^2, \ldots, \omega_n^{n-1}$ (c'est-à-dire pour les n racines $n$ièmes de l'unité) [3]. On peut supposer que n est une puissance de 2 sans nuire à la généralité, puisqu'une borne de degré donnée peut toujours être augmentée (il suffit d'ajouter des coefficients de valeur nulle). [4] On suppose que A est donné par ses coefficients : $a = (a_0, a_1, \ldots, a_{n-1})$. On définit les résultats y_k, pour $k = 0, 1, \ldots, n-1$, par

$$\begin{aligned} y_k &= A(\omega_n^k) \\ &= \sum_{j=0}^{n-1} a_j \omega_n^{kj} . \end{aligned} \tag{30.8}$$

Le vecteur $y = (y_0, y_1, \ldots, y_{n-1})$ est la ***transformée discrète de Fourier (DFT)*** du vecteur de coefficients $a = (a_0, a_1, \ldots, a_{n-1})$. On écrit également $y = \mathrm{DFT}_n(a)$.

c) Transformée rapide de Fourier

En utilisant une méthode connue sous le nom de ***Transformée rapide de Fourier (FFT)***, qui tire parti des propriétés particulières des racines complexes de l'unité, on peut calculer $\mathrm{DFT}_n(a)$ en temps $\Theta(n \lg n)$, contrairement au temps $\Theta(n^2)$ de la méthode directe.

La méthode FFT fait appel à une stratégie diviser-pour-régner, en se servant séparément des coefficients d'indice pair et des coefficients d'indice impair de $A(x)$ pour définir les deux nouveaux polynômes $A^{[0]}(x)$ et $A^{[1]}(x)$ de degré majoré par $n/2$:

$$\begin{aligned} A^{[0]}(x) &= a_0 + a_2 x + a_4 x^2 + \cdots + a_{n-2} x^{n/2-1} , \\ A^{[1]}(x) &= a_1 + a_3 x + a_5 x^2 + \cdots + a_{n-1} x^{n/2-1} . \end{aligned}$$

Notez que $A^{[0]}$ contient tous les coefficients d'indice pair de A (la représentation binaire de l'indice se termine par 0) et que $A^{[1]}$ contient tous les coefficients d'indice impair (la représentation binaire de l'indice se termine par 1). Il s'ensuit que

$$A(x) = A^{[0]}(x^2) + xA^{[1]}(x^2) , \tag{30.9}$$

---

(3) La longueur n est en réalité ce que nous appelions $2n$ à la section 30.1, puisque la borne du degré des deux polynômes est doublée avant l'évaluation. Dans le cas d'une multiplication de polynômes, on travaille donc en réalité avec les racines $(2n)$ièmes de l'unité.

(4) Quand on emploie la FFT pour faire du traitement du signal, il est généralement déconseillé d'ajouter des coefficients nuls pour arriver à une taille qui soit une puissance de 2, car cela tend à introduire des parasites haute fréquence. Une technique utilisée dans le traitement du signal pour cadrer sur une puissance de 2 consiste à utiliser un ***miroir***. Si n' désigne la plus petite puissance entière de 2 qui est plus grande que n, une façon de miroiter consiste à faire les affectations $a_{n+j} = a_{n-j-1}$ pour $j = 0, 1, \ldots, n' - n - 1$.

Scanned with CamScanner

de sorte que le problème consistant à évaluer $A(x)$ en $\omega_n^0, \omega_n^1, \ldots, \omega_n^{n-1}$ se ramène à

1) évaluer les polynômes de borne $n/2$ $A^{[0]}(x)$ et $A^{[1]}(x)$ aux points

$$(\omega_n^0)^2, (\omega_n^1)^2, \ldots, (\omega_n^{n-1})^2 , \tag{30.10}$$

puis

2) combiner les résultats selon le modèle de l'équation (30.9).

D'après le lemme de la bipartition, la liste des valeurs (30.10) n'est pas composée de n valeurs distinctes, mais seulement des $n/2$ racines $(n/2)$ièmes de l'unité, chaque racine apparaissant exactement deux fois. Ainsi, les polynômes $A^{[0]}$ et $A^{[1]}$ de borne $n/2$ sont évalués récursivement pour les $n/2$ racines $(n/2)$ièmes complexes de l'unité. Ces sous-problèmes ont exactement la même forme que le problème initial, mais ils sont moitié moins grands. Nous avons maintenant réussi à diviser un calcul DFT_n à n éléments en deux calculs $\text{DFT}_{n/2}$ à $n/2$ éléments. Cette décomposition est la base de l'algorithme récursif suivant, qui calcule la transformée discrète de Fourier d'un vecteur à n éléments $a = (a_0, a_1, \ldots, a_{n-1})$, où n est une puissance de 2.

```
FFT-RÉCURSIVE(a)
 1  n ← longueur[a]                ▷ n est une puissance de 2.
 2  si n = 1
 3     alors retourner a
 4  ω_n ← e^{2πi/n}
 5  ω ← 1
 6  a^[0] ← (a_0, a_2, ..., a_{n-2})
 7  a^[1] ← (a_1, a_3, ..., a_{n-1})
 8  y^[0] ← FFT-RÉCURSIVE(a^[0])
 9  y^[1] ← FFT-RÉCURSIVE(a^[1])
10  pour k ← 0 à n/2 − 1
11     faire y_k ← y_k^[0] + ω y_k^[1]
12           y_{k+(n/2)} ← y_k^[0] − ω y_k^[1]
13           ω ← ω ω_n
14  retourner y                    ▷ y est supposé être un vecteur colonne.
```

La procédure FFT-RÉCURSIVE fonctionne de la manière suivante. Les lignes 2–3 représentent la base de la récursivité ; la transformée discrète de Fourier d'un seul élément est l'élément lui-même, puisque dans ce cas

$$\begin{aligned} y_0 &= a_0 \omega_1^0 \\ &= a_0 \cdot 1 \\ &= a_0 . \end{aligned}$$

Les lignes 6–7 définissent les vecteurs de coefficients des polynômes $A^{[0]}$ et $A^{[1]}$. Les lignes 4, 5 et 13 garantissent que ω est mis à jour de telle sorte qu'à chaque exécution des lignes 11–12, $\omega = \omega_n^k$. (Maintenir à jour une valeur de ω d'une itération à l'autre

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

permet de gagner du temps par rapport à un calcul ω_n^k *ex nihilo* à chaque passage dans la boucle **pour**.) Les lignes 8–9 calculent récursivement les $\mathrm{DFT}_{n/2}$, en effectuant, pour $k = 0, 1, \ldots, n/2 - 1$, les affectations

$$\begin{aligned} y_k^{[0]} &= A^{[0]}(\omega_{n/2}^k)\,, \\ y_k^{[1]} &= A^{[1]}(\omega_{n/2}^k)\,, \end{aligned}$$

ou, puisque $\omega_{n/2}^k = \omega_n^{2k}$ d'après le lemme de l'annulation,

$$\begin{aligned} y_k^{[0]} &= A^{[0]}(\omega_n^{2k})\,, \\ y_k^{[1]} &= A^{[1]}(\omega_n^{2k})\,. \end{aligned}$$

Les lignes 11–12 combinent les résultats des calculs récursifs de $\mathrm{DFT}_{n/2}$. Pour $y_0, y_1, \ldots, y_{n/2-1}$, la ligne 11 donne

$$\begin{aligned} y_k &= y_k^{[0]} + \omega_n^k y_k^{[1]} \\ &= A^{[0]}(\omega_n^{2k}) + \omega_n^k A^{[1]}(\omega_n^{2k}) \\ &= A(\omega_n^k) && \text{(d'après l'équation (30.9))}\,. \end{aligned}$$

Pour $y_{n/2}, y_{n/2+1}, \ldots, y_{n-1}$, en faisant $k = 0, 1, \ldots, n/2 - 1$, la ligne 12 donne

$$\begin{aligned} y_{k+(n/2)} &= y_k^{[0]} - \omega_n^k y_k^{[1]} \\ &= y_k^{[0]} + \omega_n^{k+(n/2)} y_k^{[1]} && (\text{car } \omega_n^{k+(n/2)} = -\omega_n^k) \\ &= A^{[0]}(\omega_n^{2k}) + \omega_n^{k+(n/2)} A^{[1]}(\omega_n^{2k}) \\ &= A^{[0]}(\omega_n^{2k+n}) + \omega_n^{k+(n/2)} A^{[1]}(\omega_n^{2k+n}) && (\text{car } \omega_n^{2k+n} = \omega_n^{2k}) \\ &= A(\omega_n^{k+(n/2)}) && \text{(d'après l'équation (30.9))}\,. \end{aligned}$$

Donc, le vecteur y retourné par FFT-RÉCURSIVE est bien la transformée discrète de Fourier du vecteur d'entrée a.

Dans la boucle **pour** des lignes 10–13, chaque valeur $y_k^{[1]}$ est multipliée par ω_n^k, pour $k = 0, 1, \ldots, n/2 - 1$. Le produit est à la fois ajouté et soustrait à $y_k^{[0]}$. Comme chaque facteur ω_n^k est utilisé dans sa forme positive et dans sa forme négative, les facteurs ω_n^k sont appelés ***facteurs tournants***.

Pour déterminer le temps d'exécution de la procédure FFT-RÉCURSIVE, on remarque que, si l'on omet les appels récursifs, chaque appel prend un temps $\Theta(n)$, où n est la longueur du vecteur d'entrée. La récurrence pour le temps d'exécution est donc

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n)\,. \end{aligned}$$

Donc, un polynôme de degré borné par n peut être évalué sur les racines $n$ièmes de l'unité en temps $\Theta(n \lg n)$ à l'aide la transformée rapide de Fourier.

Scanned with CamScanner

d) Interpolation aux racines complexes de l'unité

Complétons à présent le mécanisme de la multiplication des polynômes en montrant comment interpoler les racines complexes de l'unité par un polynôme, ce qui nous permettra de passer de la représentation par paires point-valeur à la représentation par coefficients. On interpole en écrivant la transformée discrète de Fourier sous la forme d'une équation matricielle, puis en s'intéressant à la forme de la matrice inverse.

D'après l'équation (30.4), on peut écrire la transformée discrète de Fourier comme le produit de matrice $y = V_n a$, où V_n est une matrice de Vandermonde contenant les puissances appropriées de ω_n :

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \omega_n^3 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \omega_n^6 & \cdots & \omega_n^{2(n-1)} \\ 1 & \omega_n^3 & \omega_n^6 & \omega_n^9 & \cdots & \omega_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \omega_n^{3(n-1)} & \cdots & \omega_n^{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix} .$$

La composante (k, j) de V_n est égale à ω_n^{kj}, pour $j, k = 0, 1, \ldots, n-1$, et les exposants des composantes de V_n forment une table de multiplication.

Pour l'opération inverse, que l'on écrit $a = \mathrm{DFT}_n^{-1}(y)$, on peut procéder en multipliant y par la matrice V_n^{-1}, inverse de V_n.

Théorème 30.7 *Pour $j, k = 0, 1, \ldots, n-1$, la composante (j, k) de V_n^{-1} est ω_n^{-kj}/n.*

Démonstration : On montre que $V_n^{-1} V_n = I_n$, matrice identité $n \times n$. Considérons la composante (j, j') de $V_n^{-1} V_n$:

$$\begin{aligned} [V_n^{-1} V_n]_{jj'} &= \sum_{k=0}^{n-1} (\omega_n^{-kj}/n)(\omega_n^{kj'}) \\ &= \sum_{k=0}^{n-1} \omega_n^{k(j'-j)}/n . \end{aligned}$$

Cette sommation vaut 1 si $j' = j$ et 0 dans le cas contraire, d'après le lemme de sommation (lemme 30.6). Notez qu'on s'appuie sur le fait que $-(n-1) \leqslant j' - j \leqslant n-1$, de sorte que $j' - j$ n'est pas divisible par n, pour pouvoir appliquer le lemme de sommation. □

Connaissant la matrice inverse V_n^{-1}, $\mathrm{DFT}_n^{-1}(y)$ est donnée par

$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k \omega_n^{-kj} \qquad (30.11)$$

pour $j = 0, 1, \ldots, n-1$. En comparant les équations (30.8) et (30.11), on voit que, en modifiant l'algorithme FFT de façon à intervertir les rôles de a et y, remplacer ω_n par

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

ω_n^{-1} et diviser chaque élément du résultat par n, on calcule l'inverse de la transformée discrète de Fourier (voir exercice 30.2.4). Donc, DFT_n^{-1} peut également être calculée en temps $\Theta(n \lg n)$.

Donc, en utilisant la FFT et son inverse, il est possible de faire passer un polynôme, de degré borné par n, de sa représentation par coefficients à sa représentation par paires point-valeur, et réciproquement, en temps $\Theta(n \lg n)$. Dans le cas de la multiplication des polynômes, nous avons montré la chose suivante :

Théorème 30.8 (Théorème de convolution) *Pour deux vecteurs quelconques a et b de longueur n, où n est une puissance de 2,*

$$a \otimes b = \mathrm{DFT}_{2n}^{-1}(\mathrm{DFT}_{2n}(a) \cdot \mathrm{DFT}_{2n}(b)) ,$$

où les vecteurs a et b sont complétés avec des 0 jusqu'à la longueur $2n$ et où $\cdot$ représente le produit composante par composante des deux vecteurs à $2n$ éléments.

Exercices

30.2.1 Démontrer le corollaire 30.4.

30.2.2 Calculer la transformée discrète de Fourier du vecteur $(0, 1, 2, 3)$.

30.2.3 Faire l'exercice 30.1.1 en utilisant le modèle temporel $\Theta(n \lg n)$.

30.2.4 Écrire un pseudo code permettant de calculer DFT_n^{-1} en temps $\Theta(n \lg n)$.

30.2.5 Décrire la généralisation de la procédure FFT au cas où n est une puissance de 3. Donner une récurrence pour le temps d'exécution et la résoudre.

30.2.6 ⋆ Supposons qu'au lieu d'effectuer une FFT à n éléments sur le corps des nombres complexes (pour n pair), on utilise l'anneau $\mathbf{Z}_m$ des entiers modulo m, où $m = 2^{tn/2} + 1$ et t est un entier positif arbitraire. On utilise $w = 2^t$ à la place de ω_n comme racine $n$ème principale de l'unité, modulo m. Démontrer que la transformée discrète de Fourier et son inverse sont bien définies dans ce système.

30.2.7 Étant donnée une liste de valeurs $z_0, z_1, \ldots, z_{n-1}$ (avec répétitions possibles), montrer comment trouver les coefficients d'un polynôme $P(x)$, de degré borné par n, qui s'annule uniquement en $z_0, z_1, \ldots, z_{n-1}$ (avec répétitions éventuelles). Votre procédure devra s'exécuter en temps $O(n \lg^2 n)$. (*conseil :* Le polynôme $P(x)$ a un zéro en z_j si et seulement si $P(x)$ est un multiple de $(x - z_j)$.)

Scanned with CamScanner

30.2.8 ⋆ La ***transformée chirp*** d'un vecteur $a = (a_0, a_1, \ldots, a_{n-1})$ est le vecteur $y = (y_0, y_1, \ldots, y_{n-1})$, où $y_k = \sum_{j=0}^{n-1} a_j z^{kj}$ et z est un nombre complexe quelconque. La transformée discrète de Fourier est donc un cas particulier de la transformée chirp, obtenue en prenant $z = \omega_n$. Démontrer que la transformée chirp peut être évaluée en temps $O(n \lg n)$ pour un nombre complexe z quelconque. (*Conseil :* Utiliser l'équation

$$y_k = z^{k^2/2} \sum_{j=0}^{n-1} \left(a_j z^{j^2/2}\right)\left(z^{-(k-j)^2/2}\right)$$

pour voir la transformée chirp comme une convolution.)

30.3 IMPLÉMENTATIONS EFFICACES DE LA FFT

Les applications pratiques de la transformée discrète de Fourier, comme le traitement du signal, nécessitant la vitesse la plus grande possible, cette section se propose d'étudier deux implémentations efficaces de la FFT. D'abord, on commence par examiner une version itérative de l'algorithme de la FFT qui s'exécute en $\Theta(n \lg n)$ mais possède une constante cachée dans la notation Θ plus petite que l'implémentation récursive vue à la section 30.2. Ensuite, nous utiliserons les connaissances qui nous ont conduits à l'implémentation itérative pour élaborer un circuit FFT parallèle efficace.

a) Une implémentation itérative de la FFT

Commençons par remarquer que la boucle **pour** des lignes 10–13 de FFT-RÉCURSIVE implique le calcul à deux reprises de la valeur $\omega_n^k y_k^{[1]}$. En terminologie des compilateurs, ceci s'appelle une ***sous-expression commune***. On peut modifier la boucle pour la calculer une seule fois, en la stockant dans une variable temporaire t.

```
pour k ← 0 à n/2 − 1
        faire t ← ω y_k^[1]
              y_k ← y_k^[0] + t
              y_{k+(n/2)} ← y_k^[0] − t
              ω ← ω ω_n
```

L'action de cette boucle qui multiplie le facteur tournant $\omega = \omega_n^k$ par $y_k^{[1]}$, stocke le produit dans t, puis ajoute et soustrait t de $y_k^{[0]}$, est connue sous le nom de ***opération papillon*** et est montrée schématiquement à la figure 30.3.

Voici à présent comment rendre la structure de l'algorithme FFT itérative et non plus récursive. À la figure 30.4, nous avons représenté dans une structure arborescente les vecteurs d'entrée des appels récursifs d'une invocation de FFT-RÉCURSIVE, l'appel initial s'effectuant pour $n = 8$. L'arbre contient un nœud pour chaque appel à la procédure, étiqueté par le vecteur d'entrée correspondant. Chaque invocation de FFT-RÉCURSIVE effectue deux appels récursifs, à moins qu'elle reçoive en argument un

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

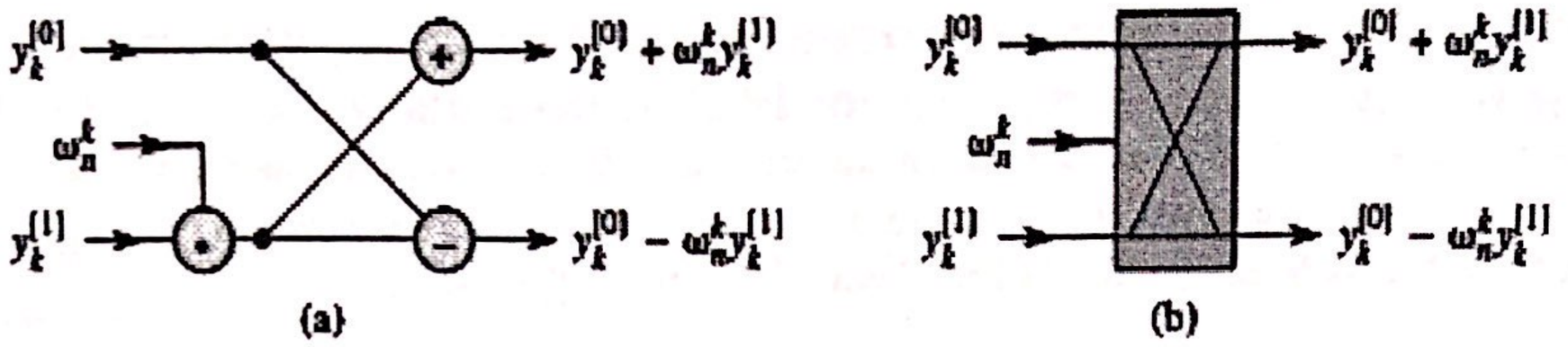


Figure 30.3 Une opération papillon. (a) Les deux valeurs d'entrée arrivent de la gauche, le facteur tournant ω_n^k est multiplié par $y_k^{[1]}$, puis la somme et la différence sont produites sur la droite. (b) Dessin simplifié d'une opération papillon. Nous utiliserons cette représentation dans un circuit FFT parallèle.

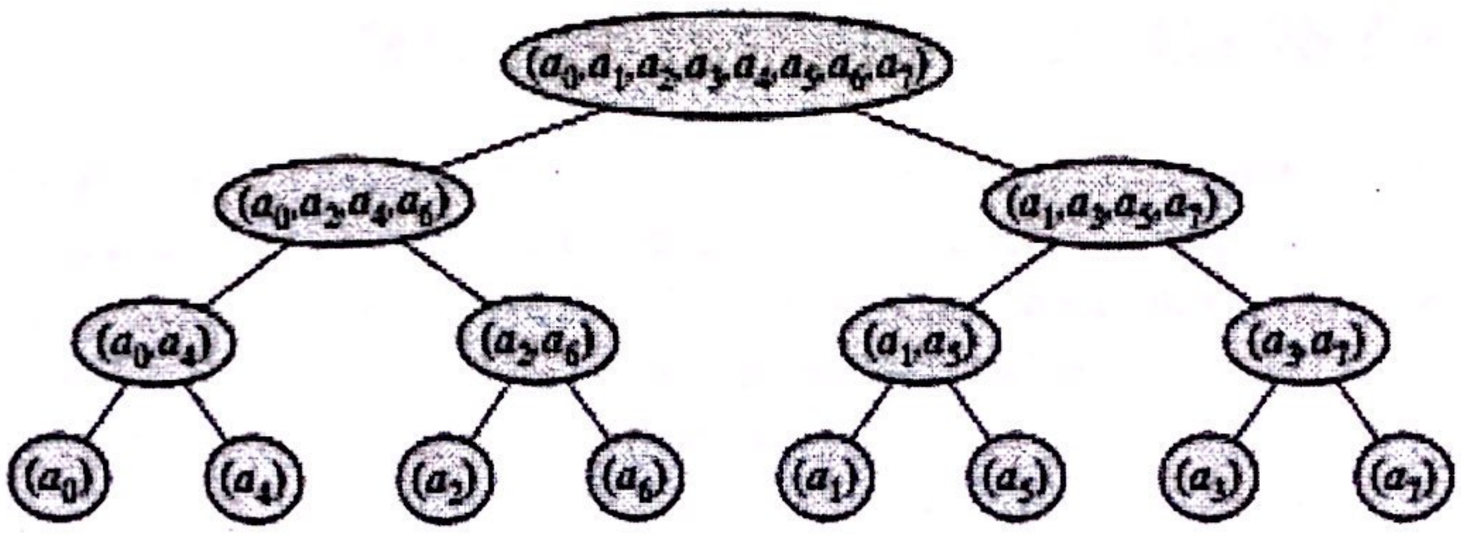


Figure 30.4 L'arborescence des vecteurs d'entrées pour les appels récursifs de la procédure FFT-RÉCURSIVE. Le premier appel se fait pour $n = 8$.

vecteur à 1 élément. Le premier appel est figuré par le fils gauche, et le second par le fils droit.

En observant cette arborescence, on remarque que, si l'on pouvait réorganiser les éléments du vecteur initial a dans leur ordre d'apparition aux feuilles, il serait possible de simuler l'exécution de la procédure FFT-RÉCURSIVE comme suit. D'abord, on prend les éléments deux par deux, on calcule la transformée discrète de Fourier de chaque paire à l'aide d'une opération papillon et on remplace la paire par sa transformée discrète de Fourier. Le vecteur contient alors $n/2$ DFT à 2 éléments. Ensuite, on prend ces $n/2$ DFT deux par deux et on calcule la transformée discrète de Fourier des quatre éléments de vecteurs qui viennent de l'exécution de deux opérations papillon, en remplaçant les deux DFT à 2 éléments par une DFT à 4 éléments. Le vecteur contient maintenant $n/4$ DFT à 4 éléments. On continue de cette manière jusqu'à ce que le vecteur contienne deux DFT à $(n/2)$ éléments, qu'on peut alors combiner à l'aide de $n/2$ opérations papillons pour donner la DFT finale à n éléments.

Pour transformer cette observation en code, on utilise un tableau $A[0..n-1]$ qui contient initialement les éléments du vecteur d'entrée a dans l'ordre où ils apparaissent dans les feuilles de l'arborescence de la figure 30.4. (On montrera plus tard comment déterminer cet ordre, connu sous le nom de permutation à inversion de bits). La combinaison devant être effectuée à chaque niveau de l'arborescence, on introduit une variable s pour compter les niveaux, qui vont de 1 (en bas, quand on combine des

Scanned with CamScanner

paires pour former des DFT à 2 éléments) à $\lg n$ (en haut, quand on combine deux DFT à $(n/2)$ éléments pour produire le résultat final). L'algorithme a donc la structure suivante :

```
1  pour s ← 1 à lg n
2      faire pour k ← 0 à n − 1 avec un pas 2^s
3          faire combiner les deux DFT à 2^(s−1) éléments
                 A[k..k + 2^(s−1) − 1] et A[k + 2^(s−1)..k + 2^s − 1]
                 pour en faire une seule DFT à 2^s éléments A[k..k + 2^s − 1]
```

On peut exprimer le corps de la boucle (ligne 3) à l'aide d'un pseudo code plus détaillé. On recopie la boucle boucle **pour** de la procédure FFT-RÉCURSIVE, en identifiant $y^{[0]}$ avec $A[k..k + 2^{s-1} - 1]$ et $y^{[1]}$ avec $A[k + 2^{s-1}..k + 2^s - 1]$. Le facteur tournant utilisé dans chaque opération papillon dépend de la valeur de s ; c'est une puissance de ω_m, où $m = 2^s$. (La variable m n'est introduite que pour une meilleure lisibilité.) On ajoute une variable temporaire u qui nous permet d'effectuer l'opération papillon sur place. Lorsqu'on remplace la ligne 3 de la structure globale par le corps de la boucle, on obtient le pseudo code suivant qui forme la base de l'implémentation parallèle que nous présenterons plus loin. Le code commence par appeler la procédure auxiliaire COPIE-INVERSION-BITS(a, A) pour copier le vecteur a dans le tableau A dans l'ordre initial dans lequel il nous faut les valeurs.

```
FFT-ITÉRATIVE(a)
 1  COPIE-INVERSION-BITS(a, A)
 2  n ← longueur[a]                ▷ n est une puissance de 2.
 3  pour s ← 1 à lg n
 4      faire m ← 2^s
 5          ω_m ← e^(2πi/m)
 6          pour k ← 0 à n − 1 avec un pas m
 7              faire ω ← 1
 8                  pour j ← 0 à m/2 − 1
 9                      faire t ← ω A[k + j + m/2]
10                          u ← A[k + j]
11                          A[k + j] ← u + t
12                          A[k + j + m/2] ← u − t
13                          ω ← ω ω_m
```

Comment COPIE-INVERSION-BITS place-t-elle les éléments du vecteur d'entrée a dans le bon ordre dans le tableau A ? L'ordre dans lequel les feuilles apparaissent sur la figure 30.4 est une ***permutation à inversion de bits***. Autrement dit, inv(k) désignant l'entier de $\lg n$ bits formé en inversant les bits de la représentation binaire de k, on souhaite placer l'élément a_k du vecteur à la position $A[\text{inv}(k)]$ dans le tableau. Sur la figure 30.4, par exemple, les feuilles apparaissent dans l'ordre $0, 4, 2, 6, 1, 5, 3, 7$; cette séquence, exprimée en binaire, est

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

000, 100, 010, 110, 001, 101, 011, 111 et si l'on renverse les bits de chaque valeur, on obtient la séquence 000, 001, 010, 011, 100, 101, 110, 111. Pour comprendre pourquoi cette transformation est souhaitable dans le cas général, on remarque qu'au niveau supérieur de l'arborescence, les indices dont le bit d'ordre inférieur est 0 sont placés dans la sous-arborescence gauche et ceux dont le bit d'ordre inférieur est égal à 1 sont placés dans la sous-arborescence de droite. En supprimant le bit de poids faible à chaque niveau, on continue ce processus tout le long de l'arbre, pour obtenir au niveau des feuilles l'ordre donné par la permutation à inversion de bits.

La fonction inv(k) étant facile à calculer, la procédure COPIE-INVERSION-BITS peut être écrite de la manière suivante.

```
COPIE-INVERSION-BITS(a, A)
1  n ← longueur[a]
2  pour k ← 0 à n − 1
3      faire A[inv(k)] ← a_k
```

L'implémentation de la FFT itérative s'exécute en $\Theta(n \lg n)$. L'appel à COPIE-INVERSION-BITS(a, A) s'exécute à coup sûr en $O(n \lg n)$, puisqu'on itère n fois et qu'on peut inverser un entier entre 0 et $n - 1$, à $\lg n$ bits, en $O(\lg n)$. (En pratique, on connaît en général à l'avance la valeur initiale de n, ce qui permet de coder une table de correspondance entre k et inv(k). De ce fait, COPIE-INVERSION-BITS peut s'exécuter en $\Theta(n)$, avec une constante cachée faible. On pourrait également utiliser l'astucieux mécanisme amorti du compteur binaire inverse décrit au problème 17.1.) Pour finir de prouver que FFT-ITÉRATIVE s'exécute en $\Theta(n \lg n)$, on va montrer que $L(n)$, nombre d'exécutions de la boucle interne (lignes 8–13), est $\Theta(n \lg n)$. La boucle **pour** des lignes 6–13 est exécutée $n/m = n/2^s$ fois pour chaque valeur de s, et la boucle la plus interne des lignes 8–13 est effectuée $m/2 = 2^{s-1}$ fois. D'où

$$
\begin{aligned}
L(n) &= \sum_{s=1}^{\lg n} \sum_{j=0}^{2^{s-1}-1} \frac{n}{2^s} \\
&= \sum_{s=1}^{\lg n} \frac{n}{2^s} \cdot 2^{s-1} \\
&= \sum_{s=1}^{\lg n} \frac{n}{2} \\
&= \Theta(n \lg n) .
\end{aligned}
$$

Scanned with CamScanner

b) Un circuit FFT parallèle

Il est possible d'exploiter une grande partie des propriétés qui nous ont permis d'implémenter un algorithme FFT itératif efficace, pour produire un algorithme parallèle efficace pour la FFT.

Nous représenterons l'algorithme parallèle FFT par un circuit qui rappelle beaucoup les réseaux de comparaison du chapitre 27. Au lieu de comparateurs, le circuit FFT emploie des opérations papillon, comme indiqué sur la figure 30.3(b). La notion de profondeur, introduite au chapitre 27, s'applique encore ici. Le circuit FFT-PARALLÈLE qui calcule la FFT sur n entrées est montré à la figure 30.5 pour $n = 8$. Le circuit commence par une permutation des entrées à base d'inversion de bits, suivie de $\lg n$ paliers, chacun étant constitué de $n/2$ opérations papillon exécutées en parallèle. La profondeur du circuit est donc $\Theta(\lg n)$.

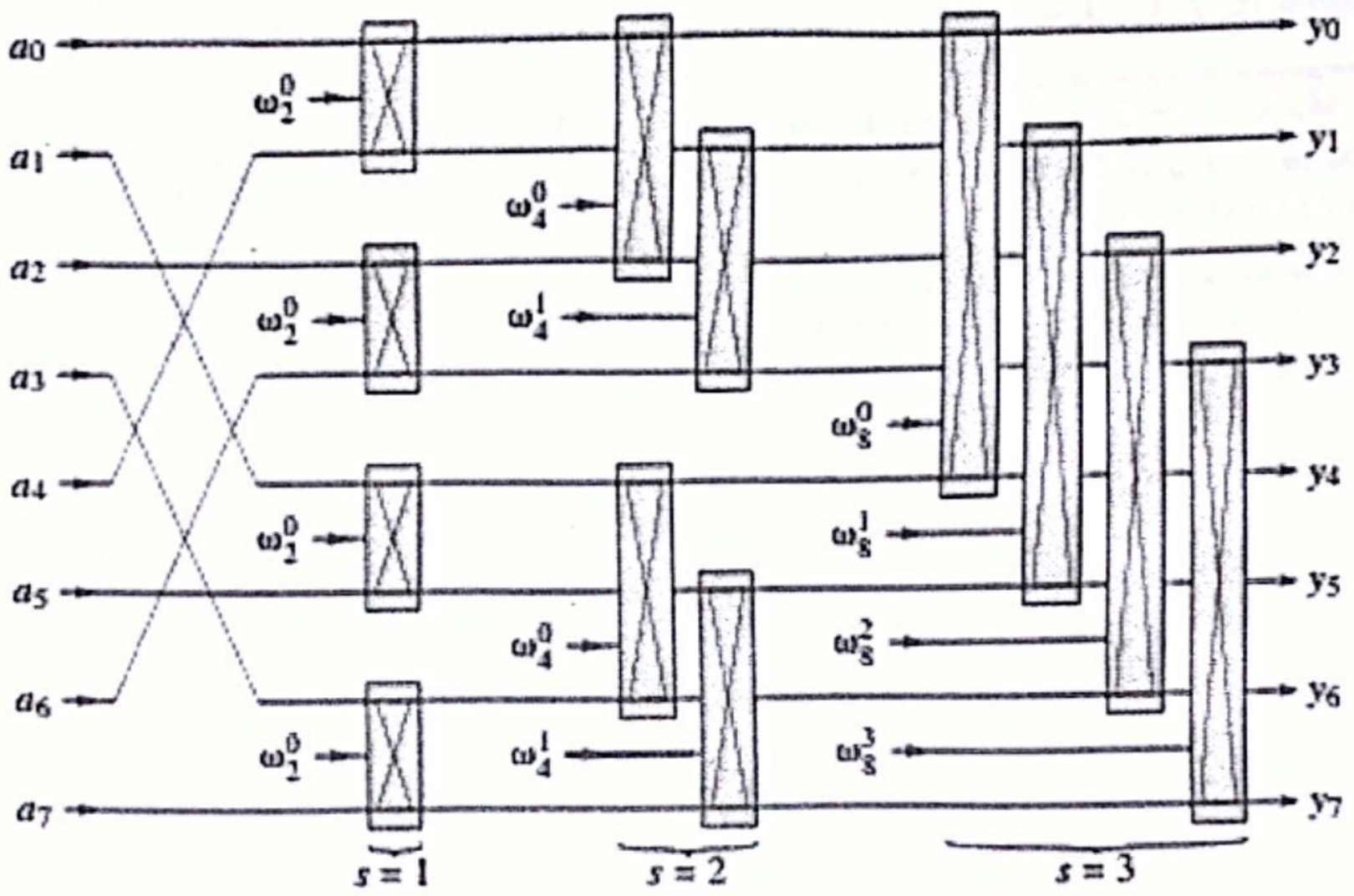


Figure 30.5 Un circuit FFT-PARALLÈLE qui calcule la FFT, ici sur $n = 8$ entrées. Chaque opération papillon prend en entrée les valeurs sur deux fils, plus un facteur tournant, et produit en sortie les valeurs sur deux fils. Les étapes des papillons sont étiquetées de façon à refléter les itérations de la boucle la plus externe de la procédure FFT-ITÉRATIVE. Seuls les fils du haut et du bas qui passent à travers un papillon interagissent avec lui ; les fils qui passent au milieu d'un papillon n'affectent pas le papillon, et réciproquement le papillon ne modifie pas leurs valeurs. Par exemple, le papillon du haut de l'étape 2 n'a rien à faire du fil 1 (fil dont la sortie est étiquetée y_1) ; ses entrées et sorties sont uniquement sur les fils 0 et 2(intitulés y_0 et y_2). Une FFT à n entrées peut être calculée sur une profondeur en $\Theta(\lg n)$ avec $\Theta(n \lg n)$ opérations papillon.

La partie la plus à gauche du circuit FFT-PARALLÈLE effectue la permutation par inversion de bits, et le reste imite la procédure itérative FFT-ITÉRATIVE. Comme chaque itération de la boucle **pour** la plus externe effectue $n/2$ opérations papillon

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

indépendantes, le circuit les fait en parallèle. La valeur de s dans chaque itération dans FFT-ITÉRATIVE correspond à un palier de papillons montré sur la figure 30.5. A l'intérieur d'un palier s, pour $s = 1, 2, \ldots, \lg n$, on a $n/2^s$ groupes de papillons (correspondant à chaque valeur de k dans FFT-ITÉRATIVE), avec 2^{s-1} papillons par groupe (correspondant à chaque valeur de j dans FFT-ITÉRATIVE). Les papillons montrés sur la figure 30.5 correspondent aux opérations papillon de la boucle la plus interne (lignes 9 –12 de FFT-ITÉRATIVE). Notez aussi que les facteurs tournants utilisés dans les papillons correspondent à ceux utilisés dans FFT-ITÉRATIVE : au palier s, on utilise $\omega_m^0, \omega_m^1, \ldots, \omega_m^{m/2-1}$, où $m = 2^s$.

Exercices

30.3.1 Montrer comment FFT-ITÉRATIVE calcule la transformée discrète de Fourier du vecteur d'entrée $(0, 2, 3, -1, 4, 5, 7, 9)$.

30.3.2 Montrer comment implémenter un algorithme FFT qui effectue la permutation par inversion de bits à la fin, et non au début, du calcul. (*conseil :* Considérer la transformée discrète de Fourier inverse.)

30.3.3 Combien de fois FFT-ITÉRATIVE calcule-t-elle des facteurs tournants à chaque étape ? Réécrire FFT-ITÉRATIVE pour qu'elle ne calcule des facteurs tournants que 2^{s-1} fois à l'étape stage s.

30.3.4 ⋆ Supposons que les additionneurs internes aux opérations papillon du circuit FFT aient parfois un comportement erratique, tel qu'ils produisent toujours un résultat nul, indépendamment de la valeurs de leurs entrées. Supposons qu'un additionneur et un seul ait ce type de comportement, mais qu'on ne sache pas lequel. Décrire une façon d'identifier l'additionneur fautif en fournissant des entrées au circuit FFT global et en observant les résultats. Quelle est l'efficacité de cette méthode ?

PROBLÈMES

30.1. Multiplication diviser-pour-régner

a. Montrer comment multiplier deux polynômes linéaires $ax + b$ et $cx + d$ à l'aide de trois multiplications seulement. (*conseil :* L'une des multiplications est $(a + b) \cdot (c + d)$.)

b. Donner deux algorithmes diviser-pour-régner permettant de multiplier deux polynômes de degré borné par n en temps $\Theta(n^{\lg 3})$. Le premier algorithme devra diviser les coefficients des polynômes en deux moitiés, l'une supérieure et l'autre inférieure ; le second algorithme devra séparer les coefficients selon la parité des indices.

Scanned with CamScanner

c. Montrer que deux entiers à n bits peuvent être multipliés en $O(n^{\lg 3})$ étapes, où chaque étape s'exécute sur au plus un nombre constant de valeurs à 1 bit.

30.2. Matrices de Toeplitz

Une ***matrice de Toeplitz*** est une matrice $n \times n$ $A = (a_{ij})$ telle que $a_{ij} = a_{i-1,j-1}$ pour $i = 2, 3, \ldots, n$ et $j = 2, 3, \ldots, n$.

a. La somme de deux matrices de Toeplitz est-elle nécessairement une matrice de Toeplitz ? Et le produit ?

b. Décrire une manière de représenter une matrice de Toeplitz de sorte que deux matrices de Toeplitz $n \times n$ puissent être additionnées en temps $O(n)$.

c. Donner un algorithme à temps $O(n \lg n)$ pour la multiplication d'une matrice $n \times n$ de Toeplitz par un vecteur de longueur n. Utiliser votre représentation de la partie (b).

d. Donner un algorithme efficace permettant de multiplier deux matrices $n \times n$ de Toeplitz. Analyser son temps d'exécution.

30.3. Transformée rapide de Fourier multidimensionnelle

On peut généraliser la transformée discrète de Fourier monodimensionnelle, définie par l'équation (30.8), à d dimensions. L'entrée est un tableau à d-dimensions $A = (a_{i_1, i_2, \ldots, i_d})$ dont les dimensions sont $n_1, n_2, \ldots, n_d$, avec $n_1 n_2 \cdots n_d = n$. On définit la transformée discrète de Fourier à d dimensions par l'équation

$$y_{k_1, k_2, \ldots, k_d} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} \cdots \sum_{j_d=0}^{n_d-1} a_{j_1, j_2, \ldots, j_d} \omega_{n_1}^{j_1 k_1} \omega_{n_2}^{j_2 k_2} \cdots \omega_{n_d}^{j_d k_d}$$

pour $0 \leqslant k_1 < n_1, 0 \leqslant k_2 < n_2, \ldots, 0 \leqslant k_d < n_d$.

a. Montrer que l'on peut calculer une DFT à d dimensions en calculant des DFT monodimensionnelles sur chaque dimension prise à tour de rôle. Plus précisément, on commence par calculer n/n_1 DFT monodimensionnelles distinctes pour la dimension 1. Ensuite, en utilisant comme entrée le résultat de la DFT pour la dimension 1, on calcule n/n_2 DFT monodimensionnelles distinctes pour la dimension 2. En prenant ce résultat comme entrée, on calcule n/n_3 DFT monodimensionnelles distinctes pour la dimension 3, etc. jusqu'à la dimension d.

b. Montrer que l'ordre des dimensions ne compte pas, de sorte que l'on peut calculer une DFT à d dimensions en calculant les DFT monodimensionnelles dans n'importe quel ordre des d dimensions.

c. Montrer que, si l'on calcule chaque DFT monodimensionnelle en calculant la transformée rapide de Fourier, le temps total mis pour calculer une DFT à d dimensions est $O(n \lg n)$, indépendamment de d.

© Dunod – La photocopie non autorisée est un délit

Scanned with CamScanner

30.4. Évaluation de toutes les dérivées d'un polynôme en un point

Étant donné un polynôme $A(x)$ de degré borné par n, sa dérivée $t$ième est définie par

$$A^{(t)}(x) = \begin{cases} A(x) & \text{si } t = 0\,, \\ \frac{d}{dx}A^{(t-1)}(x) & \text{si } 1 \leqslant t \leqslant n-1\,, \\ 0 & \text{si } t \geqslant n\,. \end{cases}$$

A partir de la représentation par coefficients $(a_0, a_1, \ldots, a_{n-1})$ de $A(x)$ et à partir d'un point donné x_0, on souhaite déterminer $A^{(t)}(x_0)$ pour $t = 0, 1, \ldots, n-1$.

a. Connaissant des coefficients $b_0, b_1, \ldots, b_{n-1}$ tels que

$$A(x) = \sum_{j=0}^{n-1} b_j (x - x_0)^j\,,$$

montrer comment calculer $A^{(t)}(x_0)$, pour $t = 0, 1, \ldots, n-1$, en temps $O(n)$.

b. Expliquer comment trouver $b_0, b_1, \ldots, b_{n-1}$ en temps $O(n \lg n)$, connaissant $A(x_0 + \omega_n^k)$ pour $k = 0, 1, \ldots, n-1$.

c. Démontrer que

$$A(x_0 + \omega_n^k) = \sum_{r=0}^{n-1} \left(\frac{\omega_n^{kr}}{r!} \sum_{j=0}^{n-1} f(j) g(r-j) \right)\,,$$

où $f(j) = a_j \cdot j!$ et

$$g(l) = \begin{cases} x_0^{-l}/(-l)! & \text{si } -(n-1) \leqslant l \leqslant 0\,, \\ 0 & \text{si } 1 \leqslant l \leqslant (n-1)\,. \end{cases}$$

d. Expliquer comment évaluer $A(x_0 + \omega_n^k)$ pour $k = 0, 1, \ldots, n-1$ en temps $O(n \lg n)$. En conclure que toutes les dérivées non triviales de $A(x)$ peuvent être évaluées en x_0 en temps $O(n \lg n)$.

30.5. Évaluation polynomiale en des points multiples

Nous avons observé que le problème consistant à évaluer un polynôme de degré majoré par $n-1$ en un point unique pouvait être résolu en temps $O(n)$, à l'aide de la règle de Horner. Nous avons découvert également qu'un tel polynôme pouvait être évalué sur toutes les n racines complexes de l'unité en temps $O(n \lg n)$ à l'aide de la FFT. Nous allons à présent montrer comment évaluer en n points arbitraires, et en temps $O(n \lg^2 n)$, un polynôme de degré majoré par n.

Pour cela, on utilisera le fait qu'il est possible de calculer en temps $O(n \lg n)$ le reste du polynôme quand un tel polynôme est divisé par un autre, résultat que nous admettrons sans le démontrer. Par exemple, le reste de la division de $3x^3 + x^2 - 3x + 1$ par $x^2 + x + 2$ est

$$(3x^3 + x^2 - 3x + 1) \bmod (x^2 + x + 2) = -7x + 5\,.$$

Scanned with CamScanner

Étant donnée la représentation par coefficients d'un polynôme $A(x) = \sum_{k=0}^{n-1} a_k x^k$ et n points $x_0, x_1, \ldots, x_{n-1}$, on désire calculer les n valeurs $A(x_0), A(x_1), \ldots, A(x_{n-1})$. Pour $0 \leqslant i \leqslant j \leqslant n-1$, on définit les polynômes $P_{ij}(x) = \prod_{k=i}^{j}(x - x_k)$ et $Q_{ij}(x) = A(x) \bmod P_{ij}(x)$. Notez que $Q_{ij}(x)$ a un degré au plus égal à $j - i$.

a. Démontrer que $A(x) \bmod (x - z) = A(z)$ pour un point z quelconque.

b. Démontrer que $Q_{kk}(x) = A(x_k)$ et que $Q_{0,n-1}(x) = A(x)$.

c. Démontrer que, pour $i \leqslant k \leqslant j$, on a $Q_{ik}(x) = Q_{ij}(x) \bmod P_{ik}(x)$ et $Q_{kj}(x) = Q_{ij}(x) \bmod P_{kj}(x)$.

d. Donner un algorithme à temps $O(n \lg^2 n)$ pour évaluer $A(x_0), A(x_1), \ldots, A(x_{n-1})$.

30.6. FFT et arithmétique modulaire

La manière dont nous avons défini la transformation discrète de Fourier impose l'utilisation des nombres complexes, ce qui peut conduire à une perte de précision due aux erreurs d'arrondis. Pour certains problèmes, on sait à l'avance que la réponse ne contiendra que des entiers et il est alors souhaitable d'utiliser une variante de la FFT basée sur l'arithmétique des congruences, de façon à être sûr que la réponse est calculée exactement. Un exemple de ce type de problème serait, par exemple, de multiplier deux polynômes à coefficients entiers. Une approche est donné à l'exercice 30.2.6, faisant appel à un modulo de longueur $\Omega(n)$ bits pour gérer une transformée discrète de Fourier sur n points. Le problème que voici propose une autre approche qui utilise un modulo de longueur plus raisonnable $O(\lg n)$; il impose que soient comprises les notions du chapitre 31.

Soit n une puissance de 2.

a. On suppose que l'on recherche le plus petit k tel que $p = kn+1$ est premier. Donner une démonstration heuristique simple qui justifie le fait que l'on peut s'attendre à ce que k vaille environ $\lg n$. (La valeur de k peut être beaucoup plus grande ou beaucoup plus petite, mais on peut raisonnablement espérer examiner $O(\lg n)$ valeurs candidates de k en moyenne.) Comparer la longueur attendue de p à la longueur de n.
Soit g un générateur de $\mathbb{Z}_p^*$ et soit $w = g^k \bmod p$.

b. Dire pourquoi la transformée discrète de Fourier et son inverse sont des opérations inverses bien définies modulo p, w étant utilisée comme racine $n$ème principale de l'unité.

c. Prouver que la FFT et son inverse peuvent fonctionner modulo p en temps $O(n \lg n)$, en supposant que les opérations sur des mots de $O(\lg n)$ bits prennent un temps unitaire. On suppose que l'algorithme a comme paramètres p et w.

d. Calculer la transformée discrète de Fourier modulo $p = 17$ du vecteur $(0, 5, 3, 7, 7, 2, 1, 6)$. Notez que $g = 3$ est un générateur de $\mathbb{Z}_{17}^*$.

© Dunod - La photocopie non autorisée est un délit

Scanned with CamScanner

NOTES

Le livre de VanLoan [303] est une référence en matière de présentation de la transformation rapide de Fourier. Press, Flannery, Teukolsky et Vetterling [248, 249] contient une bonne description de la transformation rapide de Fourier et de ses applications. Pour une excellente introduction au traitement du signal, domaine d'application célèbre de la FFT, voir les livres de Oppenheim et Schafer [232] et de Oppenheim et Willsky [233]. Le manuel de Oppenheim et Schafer montre aussi comment traiter les cas où n n'est pas une puissance entière de 2.

L'analyse de Fourier ne se limite pas aux données à 1 dimension. Elle sert énormément, dans le domaine du traitement d'image, pour analyser des données à 2 ou plusieurs dimensions. Les livres de Gonzalez et Woods [127] et Pratt [246] traitent des transformations de Fourier multidimensionnelles et de leurs applications au traitement d'image, alors que les manuels de Tolimieri, An et Lu [300] et Van Loan [303] traitent des mathématiques des transformations rapides de Fourier multidimensionnelles.

On attribue le plus souvent à Cooley et à Tukey [68] l'élaboration de la FFT dans les années 1960. La FFT a, en réalité, été découverte de nombreuses fois auparavant, mais son importance n'a été pleinement comprise qu'avec l'avènement des ordinateurs. Press, Flannery, Teukolsky et Vetterling attribuent les origines de la méthode à Runge et à König en 1924, mais un article de Heideman, Johnson et Burrus [141] fait remonter l'histoire de la FFT jusqu'à C. F. Gauss en 1805.

Scanned with CamScanner